

02 Functions and built-in functions in Python

Introduction to Python – RSE course

Function? Why and what?

- *breaks* complex problems into smaller parts
- enables *re-use*
- makes code more *readable*

```
# Function with return value
def add_numbers(a, b):
    result = a + b
    return result
```

→ DEFINING A FUNCTION

- Begin the definition of a new function `def`.
- Followed by the name of the function.
- Then *parameters* in parentheses.
 - Empty parentheses if the function doesn't take any inputs.
- Then a colon and then an indented block of code
- Use `return ...` to give a value back to the caller
- The function doesn't need a return

Calling a function

- **function_name** – the name of the function
- **arguments** – values we pass to the function

```
# Function with return value
def add_numbers(a, b):
    result = a + b
    return result

sum_result = add_numbers(5, 3)
print(f"Sum: {sum_result}")
```

Defining a function **does not** run it.

- Like assigning a value to a variable.
- Must call the function to execute the code it contains

Indentation is always meaningful in Python!

```
firstName = "Jon"
lastName = "Smith"
```

```
File "<ipython-input-7-f65f2962bf9c>", line 2
    lastName = "Smith"
    ^
```

```
IndentationError: unexpected indent
```

Variable scope

Local variables

- defined *inside a function*
- exist *only inside that function*

Global variables

- defined *outside functions*
- can be accessed *throughout the program*

```
tax_rate = 0.21    # global constant

def calculate_tax(price):
    tax = price * tax_rate    # local variable
    return tax

price = 100
print(calculate_tax(price))
```

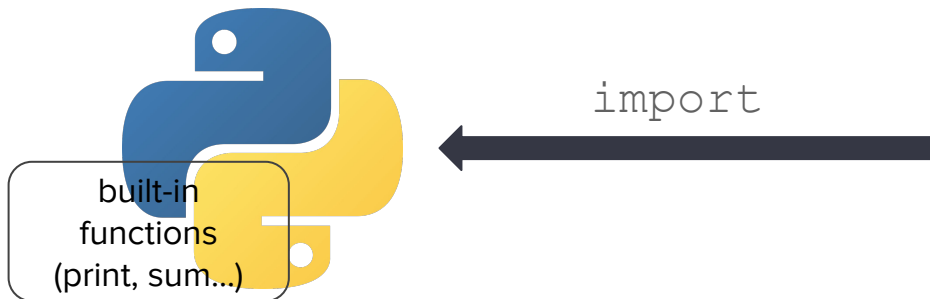
Built-in Functions

- Python already provides many **built-in functions**.
- These are functions that are **available by default** — we can use them without importing any library.
- For example:
 - a. `print()` → displays output
 - b. `type()` → returns data type of an object
 - c. `len()` → returns the length of an object
 - d. `sum()` → adds numbers in a list
 - e. `max()` / `min()` → finds largest or smallest value
 - f. `sorted()` → sorts items in list
 - g. `help()` → provides help and documentation for functions

Built-in Functions in Libraries

- Python already contains many useful **built-in functions**.
- But even more functionality is available in **libraries**.
- Libraries contain implementations of many **well-known algorithms and tools**, so you usually don't need to implement them yourself.

→ NEXT WEEK WE WILL LOOK AT LIBRARY NUMPY



Key takeaways

- functions **group instructions into reusable blocks.**
 - pass **arguments** to a function and receive a **result** using `return`.
 - variables defined inside a function are **local** and exist only within that function.
 - functions like `print()`, `len()`, `sum()` help us **write simpler and shorter code.**
-

Voluntary homework

- for you to be sure you learned something today
- try to write the code on your own and then use AI if its needed for debugging
- try to make sure you understand every line of the code
- easy **student grade analyzer**